

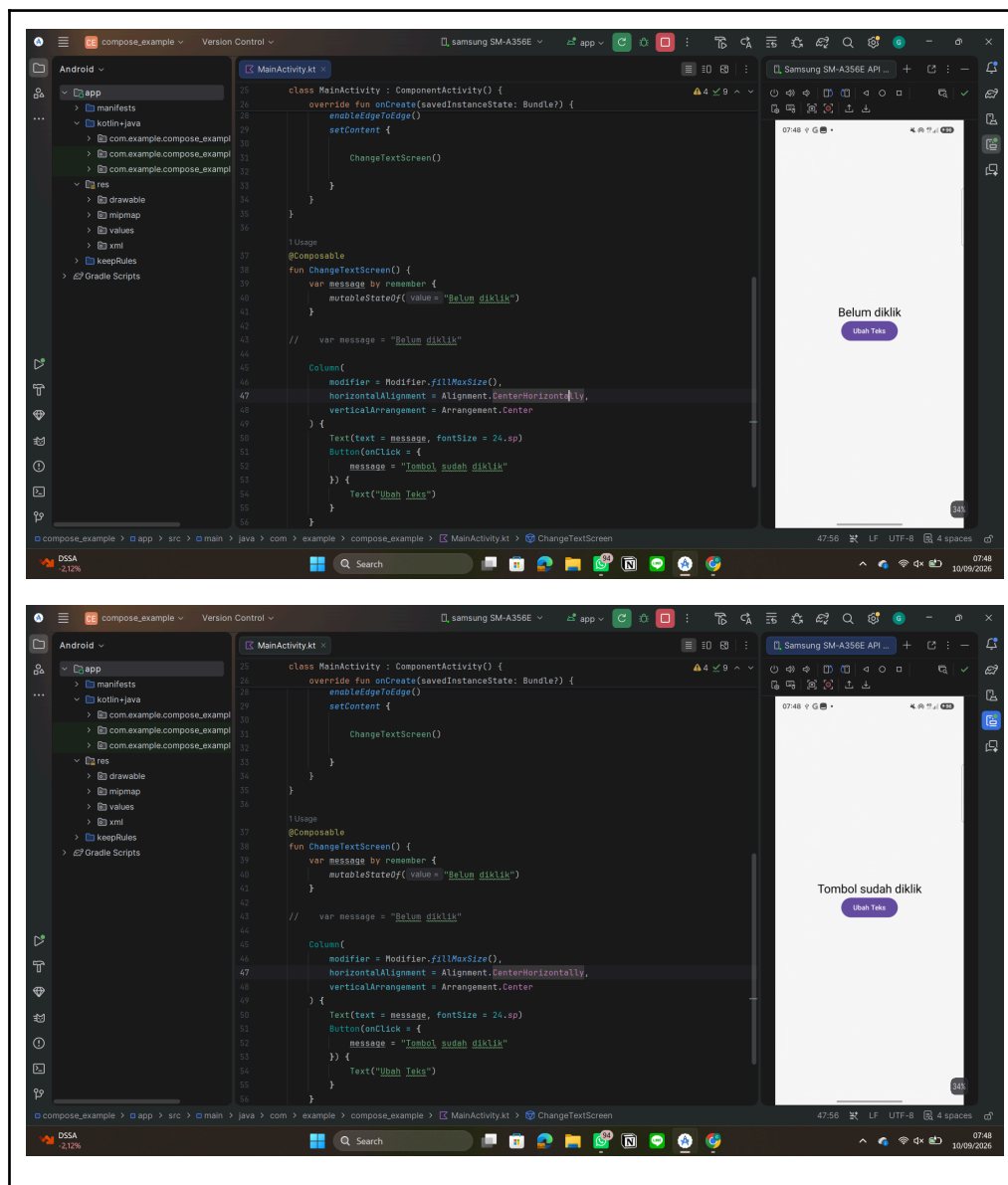
Nama : Givenia Marudur Hasiana
NIM : 245150400111051
Mata Kuliah : Pengembangan Aplikasi Mobile
Kelas : SI - F

MODUL 3

EVENT, STATE, DAN IMPLISIT INTENT PADA JETPACK COMPOSE

1. Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1.1. Screenshot



1.2. Objek yang diubah di XML/View

Yang diubah langsung adalah objek TextView (tvMessage), tepatnya properti .text miliknya. Jadi programnya "nyari" dulu si TextView pakai findViewById, baru propertinya diubah manual.

1.3. Nilai yang diubah di Compose

Yang diubah bukan objek Text-nya, tapi state-nya, yaitu variabel message. Baris kodenya:

```
message = "Tombol sudah diklik"
```

Text(text = message) cuma "membaca" nilai state ini, jadi pas state-nya berubah, otomatis Text-nya ikut update sendiri.

1.4. Kenapa tidak ada tvMessage.text atau setText() di Compose?

Karena di Compose tidak ada konsep "cari komponen lalu suruh ubah". Compose itu declarative — kita cukup bilang "tampilan ini hasil dari state ini", jadi kalau state-nya berubah, Compose yang otomatis gambar ulang UI-nya (recomposition). Kita tidak perlu megang objek View-nya secara manual kayak di XML.

1.5. Alur XML/View: event → View → perubahan tampilan

Klik tombol (event) → setOnClickListener jalan → program ambil referensi tvMessage lewat findViewById (View) → properti .text diganti manual → tampilan berubah karena View-nya sendiri yang di-update.

1.6. Alur Compose: event → state → recomposition → perubahan tampilan

Klik tombol (event) → onClick jalan → variabel message (state) diubah nilainya → Compose ngedeteksi state berubah → Compose jalanin recomposition, yaitu gambar ulang bagian UI yang pakai state itu → tampilan Text otomatis berubah sesuai nilai state terbaru.

1.7. Kalau message diganti jadi variabel biasa (var message = "Belum diklik")?

Saat tombol diklik, nilai message di baliknya memang berubah, tapi UI-nya tidak bakal ikut berubah alias tetap menampilkan teks lama. Karena Compose cuma "peka" sama perubahan yang lewat state (mutableStateOf). Variabel biasa itu tidak ke-track oleh sistem recomposition Compose, jadi walau nilainya berubah di background, Compose tidak tau harus gambar ulang UI-nya.

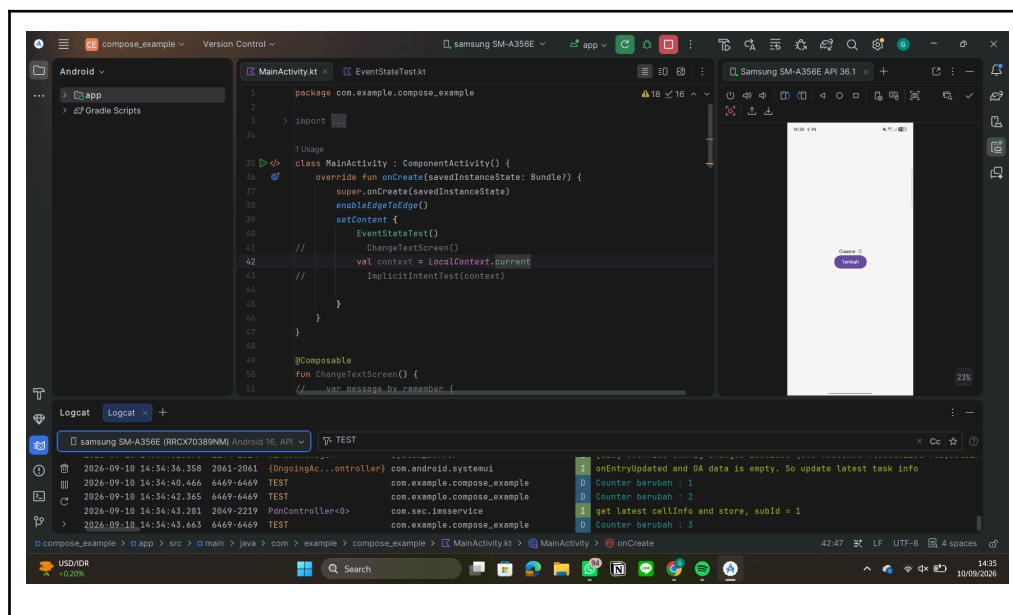
1.8. Kesimpulan perbedaan cara berpikir imperatif dan deklaratif

Pendekatan imperatif mengharuskan pengembang secara eksplisit mencari referensi komponen UI terlebih dahulu, kemudian memerintahkan komponen tersebut untuk mengubah propertinya sendiri, pengembang bertanggung

jawab penuh atas komponen mana yang diperbarui dan bagaimana caranya. Sebaliknya, pendekatan deklaratif mengharuskan pengembang cukup mendeskripsikan bagaimana seharusnya tampilan terlihat berdasarkan data (state) yang ada, sementara proses pembaruan tampilan itu sendiri diserahkan sepenuhnya kepada framework (Compose). Dengan demikian, pendekatan deklaratif memungkinkan kode menjadi lebih ringkas dan pengembang dapat lebih berfokus pada pengelolaan data, bukan pada mekanisme pembaruan UI secara manual.

2. Event Handling State

2.1. Screenshot



2.2. Yang muncul di Logcat setiap kali tombol Tambah diklik

Setiap kali tombol diklik, tetap muncul baris log baru pada Logcat. Nilai count pada log tersebut memang terus bertambah sesuai jumlah klik yang dilakukan.

2.3. Yang terjadi pada tampilan aplikasi

Meskipun nilai pada Logcat terus bertambah, tampilan teks "Counter : 0" pada layar tidak berubah dan tetap menunjukkan angka 0, meskipun tombol telah diklik berkali-kali.

2.4. Penyebab hal tersebut terjadi

Hal ini terjadi karena count pada percobaan ini hanyalah variabel Kotlin biasa, bukan state. Nilainya memang bertambah secara nyata di dalam memori (sehingga tetap terlihat pada Logcat), namun karena tidak dideklarasikan menggunakan mutableStateOf, Compose tidak memiliki mekanisme untuk

mendeteksi bahwa nilai tersebut telah berubah. Akibatnya, Compose tidak menjalankan recomposition dan tampilan tetap tidak diperbarui.

2.5. Mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.

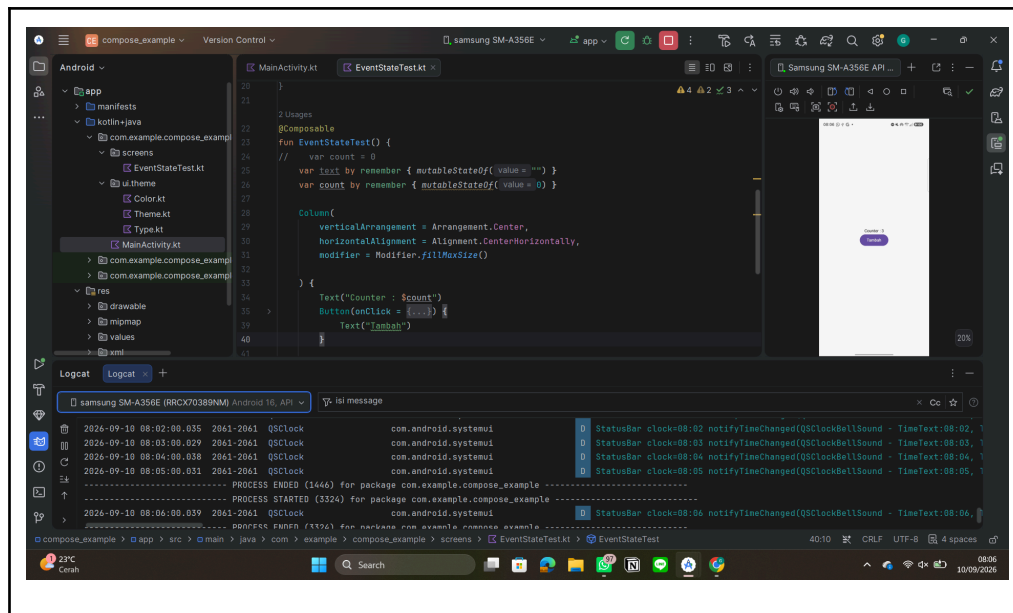
2.6. Perubahan kode dari var count = 0

2.7. Kode lengkap EventStateTest

```
@Composable
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()

    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}
```

2.8. Screenshot setelah menggunakan state



2.9. Yang muncul di Logcat (setelah menggunakan state)

Sama seperti sebelumnya, setiap klik tombol tetap memunculkan baris log baru seperti "Counter berubah : 1, 2, 3, ...".

2.10. Yang terjadi pada tampilan aplikasi

Berbeda dari percobaan sebelumnya, tampilan teks "Counter : ..." pada layar kini ikut berubah setiap kali tombol diklik, sesuai dengan nilai terbaru dari count.

2.11. Penyebab perbedaan hasil dari percobaan sebelumnya

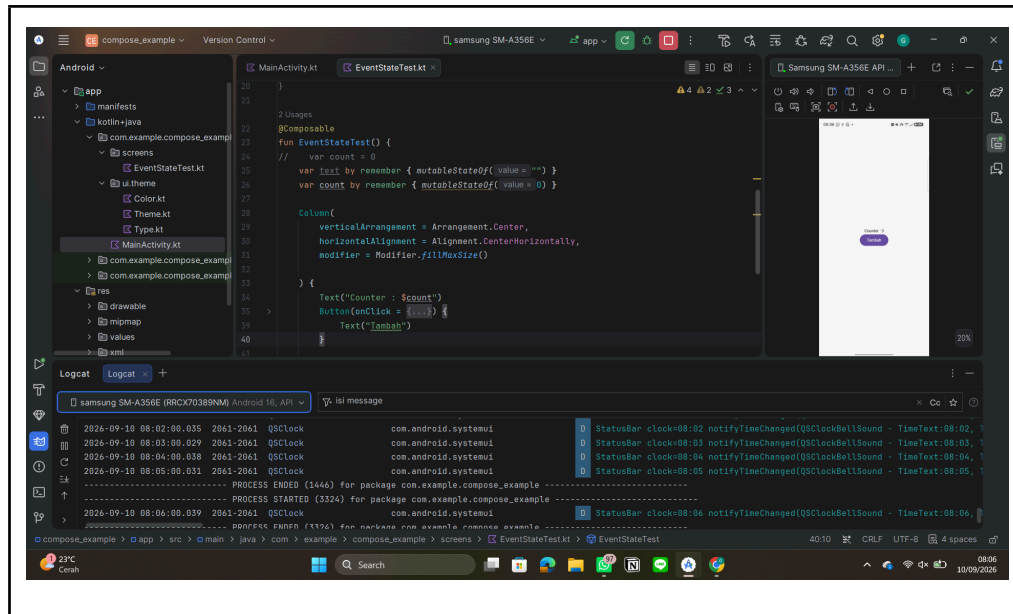
Perbedaan ini terjadi karena count sekarang telah dideklarasikan sebagai state resmi menggunakan mutableStateOf, sehingga Compose dapat mengamati (observe) setiap perubahan nilainya. Ketika nilai count berubah, Compose secara otomatis menjalankan recomposition pada bagian UI yang bergantung pada state tersebut, sehingga tampilan diperbarui secara real-time.

2.12. Pengertian recomposition pada Compose

Recomposition adalah proses di mana Compose menjalankan ulang (menggambar ulang) fungsi composable yang bergantung pada suatu state, setiap kali nilai state tersebut berubah. Proses ini bersifat selektif — hanya bagian UI yang benar-benar terpengaruh oleh perubahan state yang akan digambar ulang, bukan seluruh tampilan aplikasi — sehingga proses pembaruan UI pada Compose tetap efisien meskipun terjadi secara berulang.

2.13. Menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah

2.14. Screenshot



2.15. Yang muncul di Logcat saat menginputkan huruf pada Text Field

Setiap kali satu karakter diketik pada Text Field, muncul baris log baru. Setiap perubahan karakter tercatat secara individual pada Logcat.

2.16. Yang terjadi pada tampilan aplikasi saat menginputkan huruf

Komponen Text Field pada tampilan langsung menampilkan karakter yang diketik secara real-time, sesuai dengan input yang diberikan pengguna.

2.17. Penyebab hal tersebut terjadi

Hal ini terjadi karena variabel text, sama seperti count, dideklarasikan sebagai state menggunakan mutableStateOf. Setiap kali fungsi onValueChange dipanggil (yaitu setiap kali pengguna mengetik), nilai state text diperbarui, yang kemudian memicu Compose untuk menjalankan recomposition dan menampilkan nilai terbaru pada Text Field.

2.18. State yang terdapat pada program terakhir

Terdapat dua state, yaitu count (untuk menyimpan nilai counter) dan text (untuk menyimpan nilai input pada Text Field).

2.19. Event yang terdapat pada program terakhir

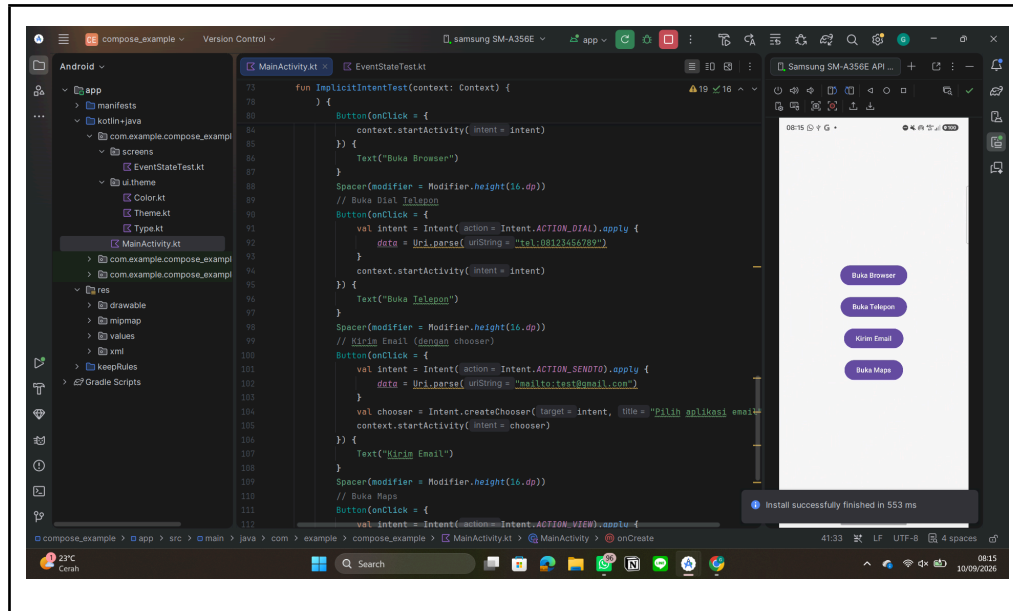
Terdapat dua event, yaitu onClick pada Button "Tambah" dan onValueChange pada OutlinedTextField.

2.20. Waktu dijalankannya event-event tersebut

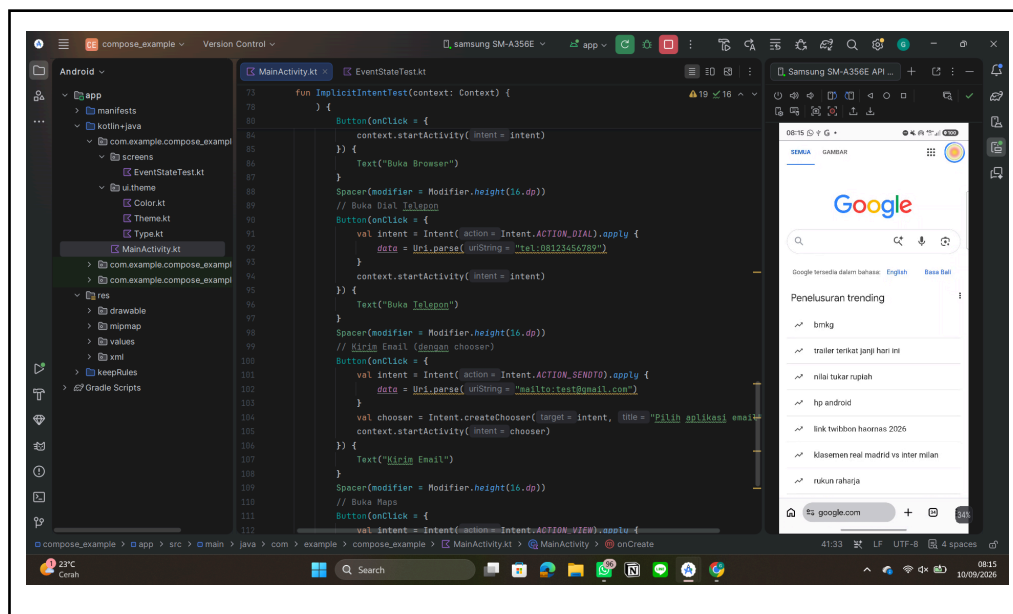
onClick dijalankan pada saat pengguna menekan (mengklik) tombol "Tambah". onValueChange dijalankan setiap kali terjadi perubahan nilai teks pada Text Field, yaitu setiap kali pengguna mengetik atau menghapus karakter pada kolom input tersebut.

3. Implisit Intent

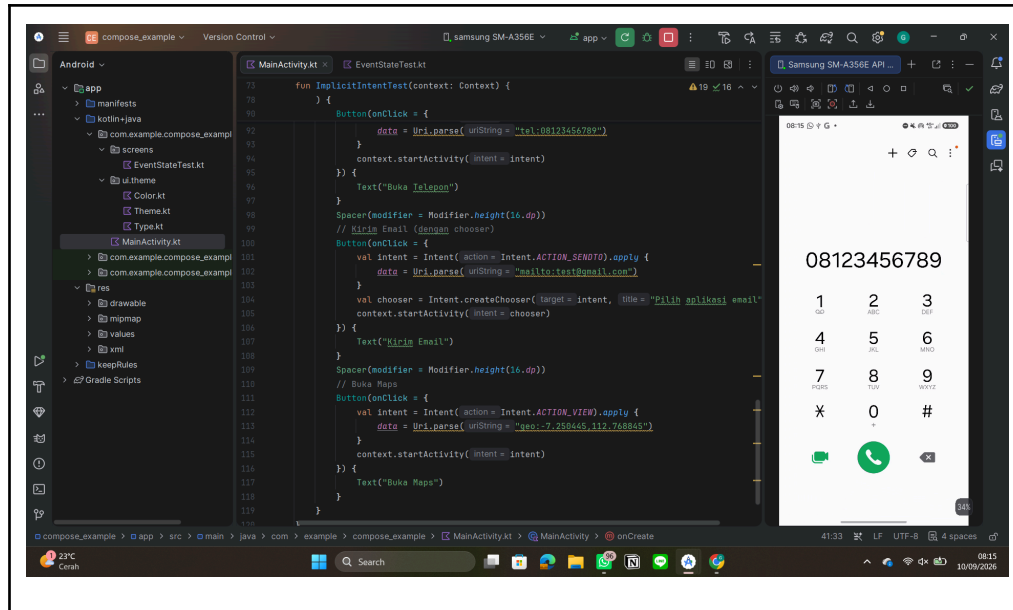
3.1. Memastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik



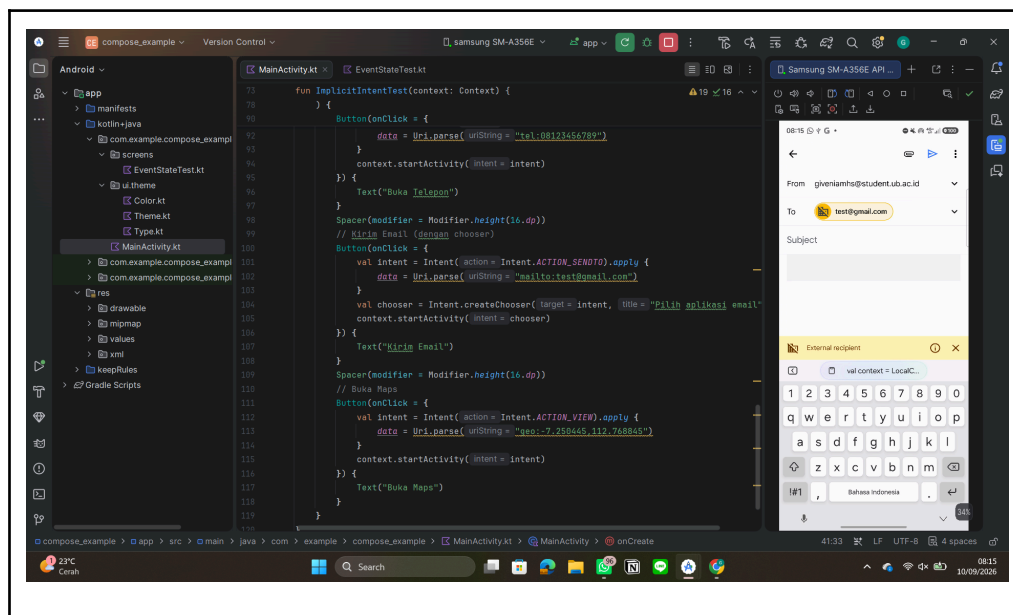
3.2. Buka Browser



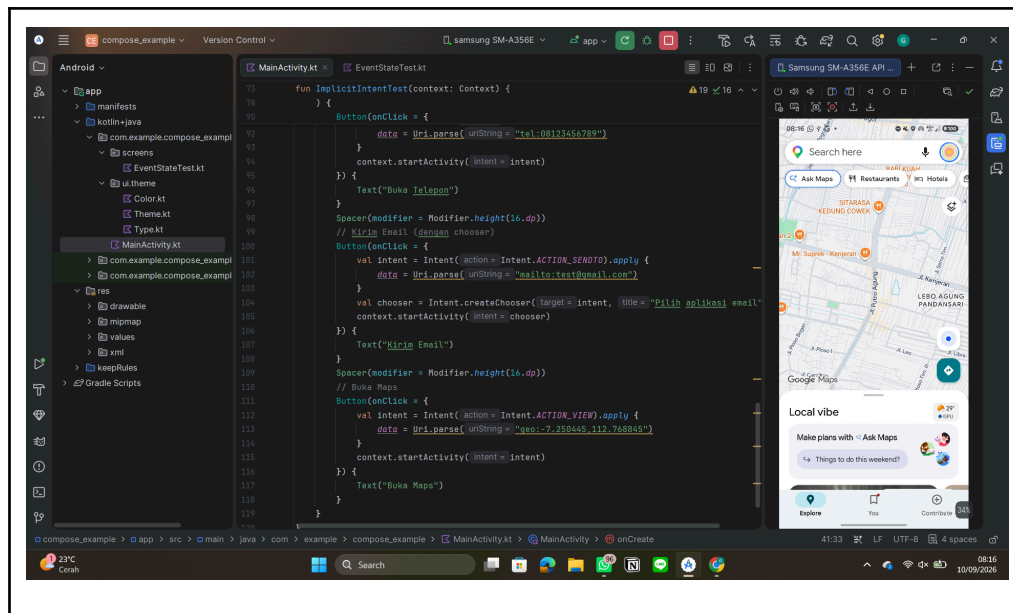
3.3. Buka Telepon



3.4. Kirim Email



3.5. Buka Maps



- 3.6. Mengamati apa yang terjadi pada setiap aksi
- 3.7. Pada kode yang digunakan, terdapat beberapa action
- 3.8. Intent.ACTION_VIEW
- 3.9. Intent.ACTION_DIAL
- 3.10. Intent.ACTION_SENDTO
- 3.11. Perbedaan fungsi masing-masing action intent
 - Intent.ACTION_VIEW — digunakan untuk menampilkan atau membuka suatu konten kepada pengguna secara umum. Action ini bersifat fleksibel; aplikasi yang terbuka bergantung pada jenis data yang disertakan, misalnya membuka browser jika data berupa URL, atau membuka aplikasi peta jika data berupa koordinat geografis.
 - Intent.ACTION_DIAL — digunakan untuk membuka aplikasi telepon dengan nomor yang sudah terisi otomatis pada layar dial, namun tidak langsung melakukan panggilan; pengguna masih harus menekan tombol panggil secara manual.
 - Intent.ACTION_SENDTO — digunakan untuk mengirimkan sesuatu ke tujuan tertentu, umumnya dipasangkan dengan skema URI seperti mailto: agar sistem mengenali bahwa Intent ini dimaksudkan untuk mengirim email, bukan sekadar menampilkan halaman.
- 3.12. Pada kode yang digunakan, terdapat beberapa data
- 3.13. data = Uri.parse("mailto:test@gmail.com")
- 3.14. data = Uri.parse("https://www.google.com")
- 3.15. data = Uri.parse("geo:-7.250445,112.768845")
- 3.16. Fungsi data pada Intent

Parameter data berisi informasi spesifik berupa URI yang menjadi target atau objek yang akan diproses oleh Intent tersebut. Data inilah yang menentukan "ke mana" atau "terhadap apa" suatu aksi (action) akan dijalankan.

3.17. Hubungan antara Action dan Data pada pilihan aplikasi yang dimunculkan sistem Android

Action menentukan jenis aksi yang ingin dilakukan (menampilkan, memanggil, mengirim, dan sebagainya), sementara Data menentukan target atau konten spesifik dari aksi tersebut. Sistem Android menggunakan kombinasi antara Action dan Data ini untuk menentukan aplikasi mana saja yang mampu menangani permintaan tersebut. Sebagai contoh, meskipun sama-sama menggunakan ACTION_VIEW, sistem akan membuka aplikasi browser apabila data berupa URL (<https://...>), namun akan membuka aplikasi peta apabila data berupa skema geografis (<geo:...>). Dengan demikian, penentuan aplikasi yang dijalankan bergantung pada kecocokan antara Action dan Data secara bersamaan, bukan hanya salah satu di antara keduanya.